

Documentación de Modelos de IA

Autores: Anthony Jiménez, Aaron Gónzales

Datasets utilizados:

BodyFat: <https://www.kaggle.com/datasets/fedesoriano/body-fat-prediction-dataset>

Bitcoin: <https://www.kaggle.com/datasets/team-ai/bitcoin-price-prediction/version/1>

Vino: <https://www.kaggle.com/datasets/rajyellow46/wine-quality>

Aguacate: <https://www.kaggle.com/datasets/neuromusic/avocado-prices>

Telecom Churn: <https://github.com/IBM/telco-customer-churn-on-icp4d/blob/master/data/Telco-Customer-Churn.csv>

Modelo: Aguacate

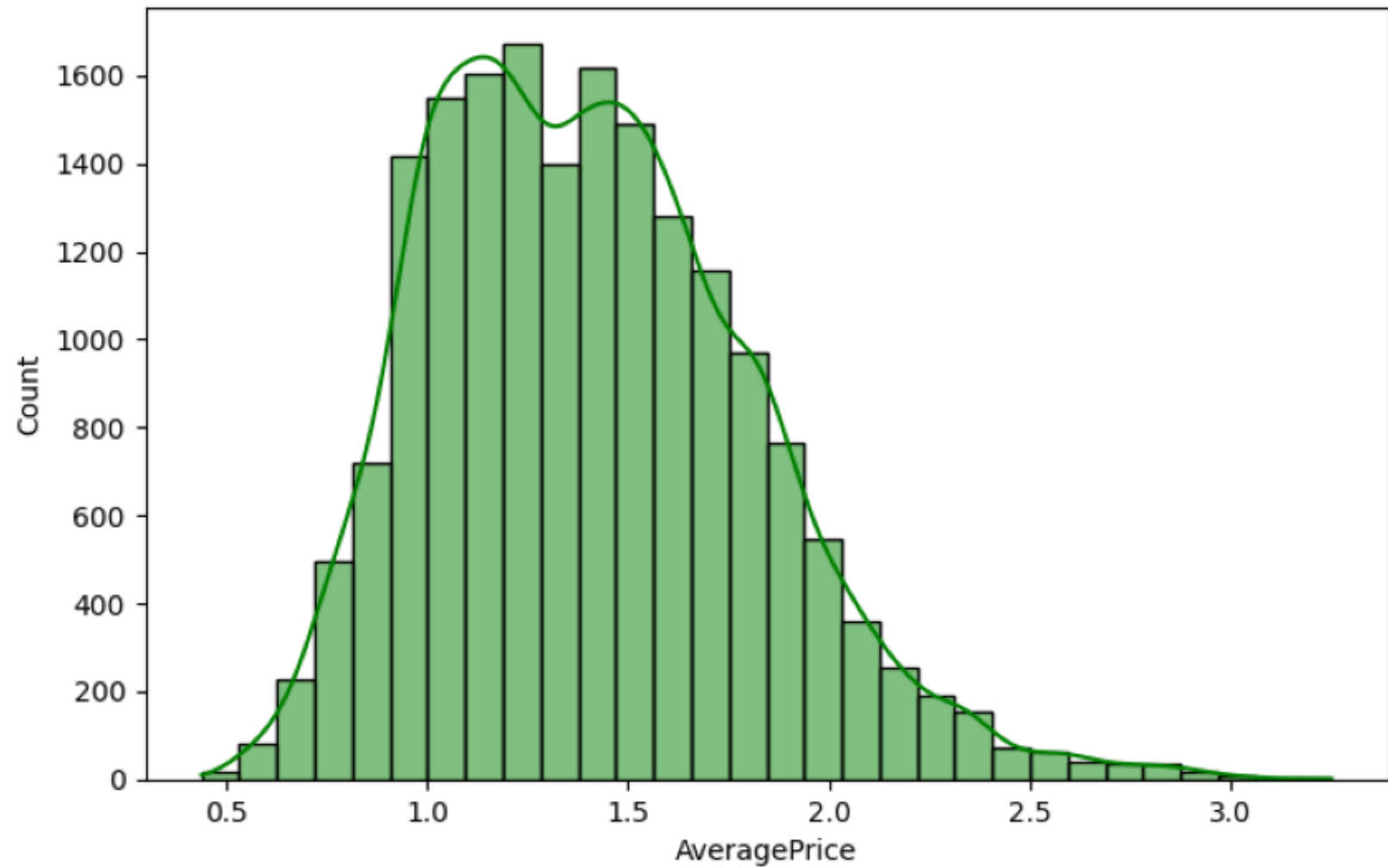
Se predice el precio promedio (AveragePrice) de los aguacates según volumen de venta, tipo, año y región. Modelo: Random Forest Regressor.

Métricas del modelo Aguacate

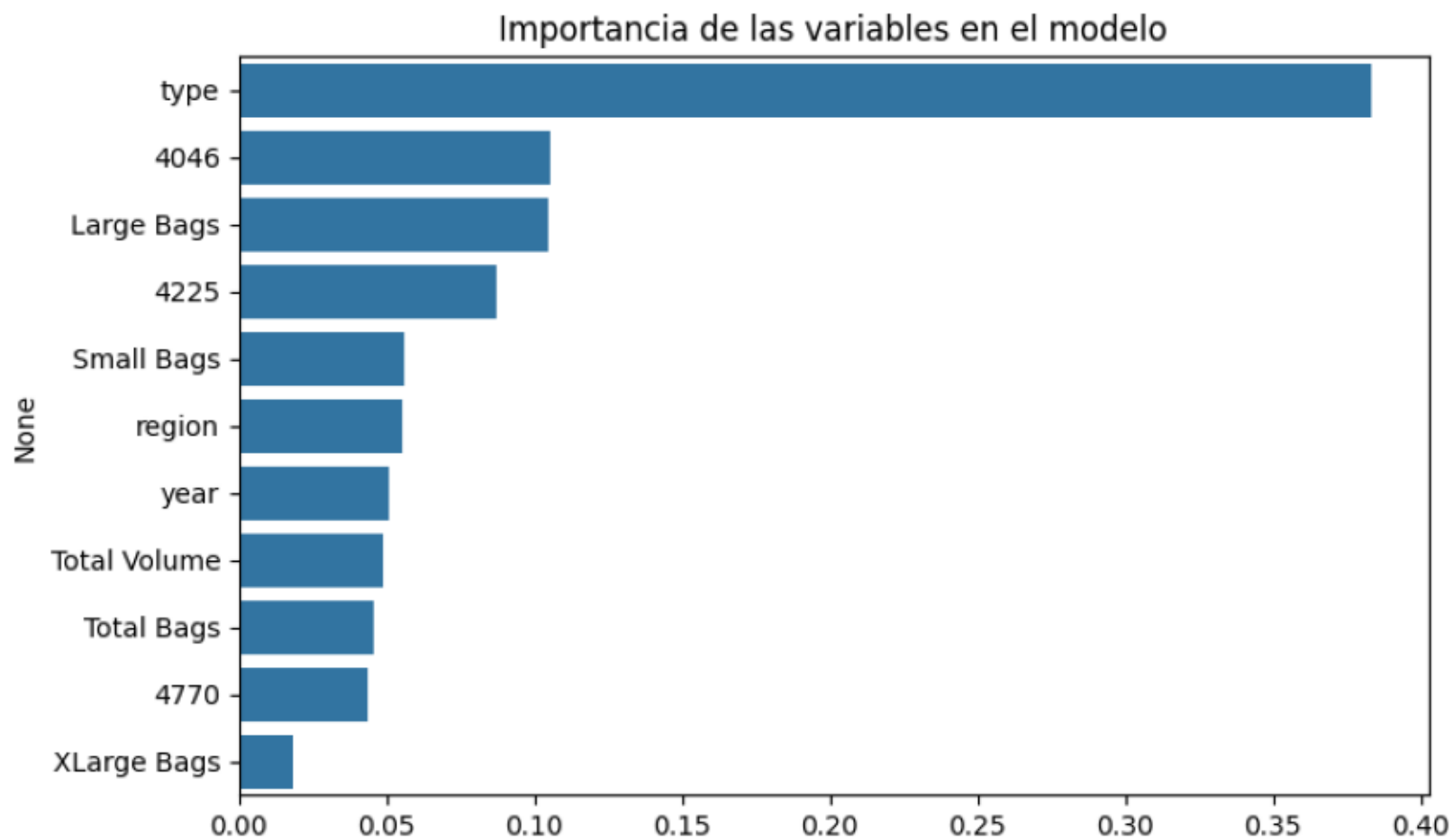
Métrica	Valor
MAE	0.1097624931506849
RMSE	0.1557360116852118
R^2	0.8490455983516202

Aguacate - averageprice_distribution.png

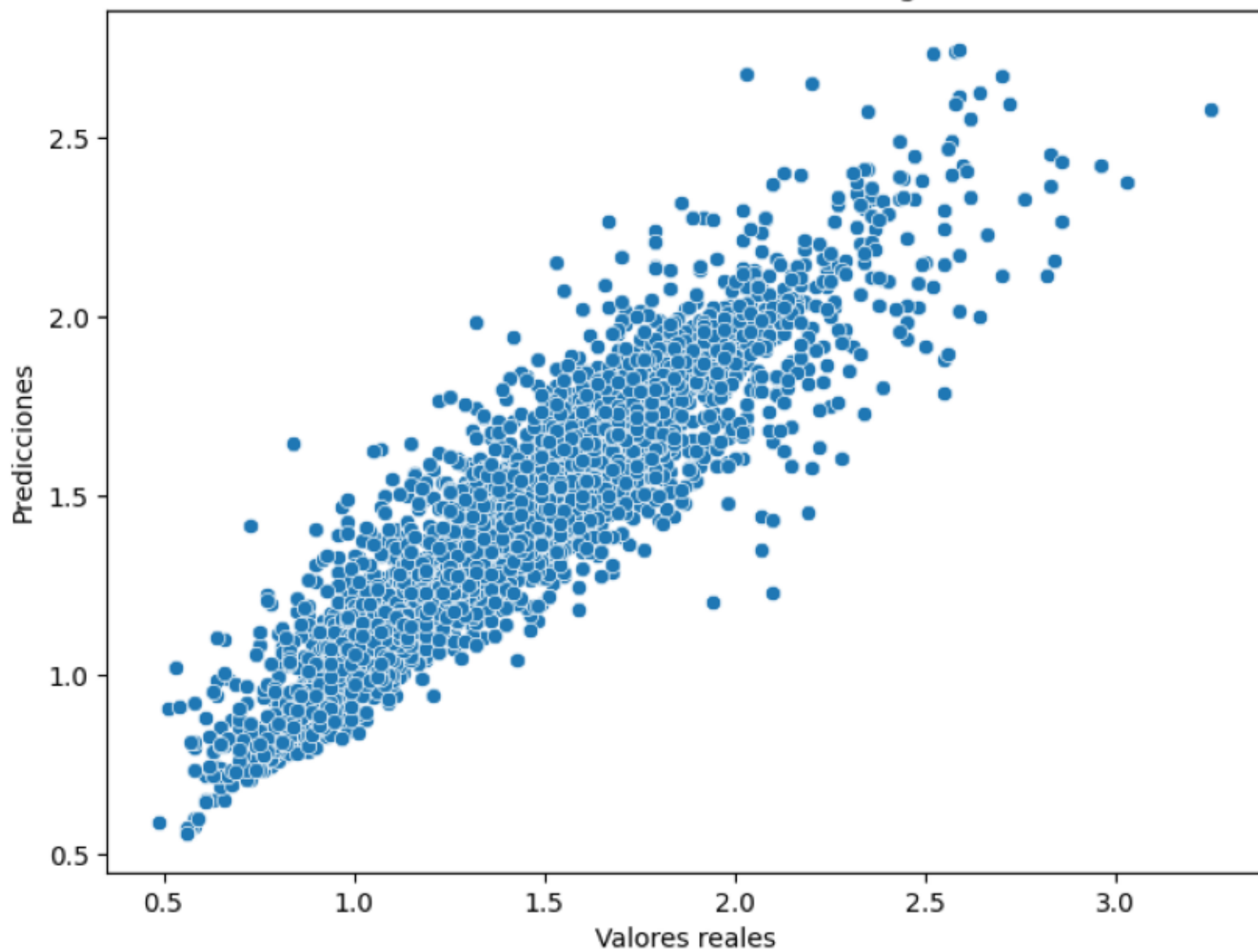
Distribución del Precio Promedio del Aguacate



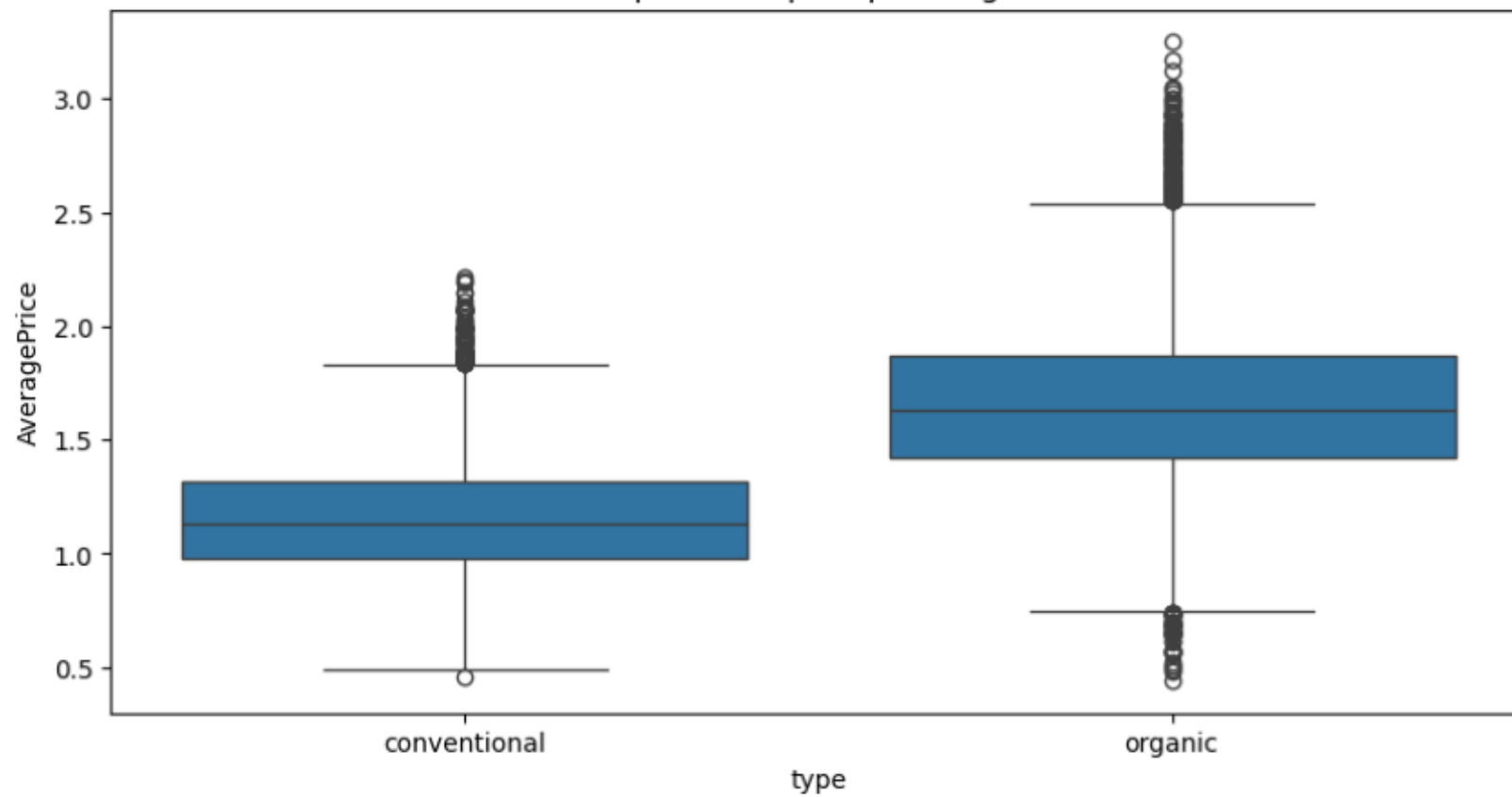
Aguacate - feature_importance.png



Aguacate - pred_vs_real.png
Predicción vs Realidad - Precio de Aguacate

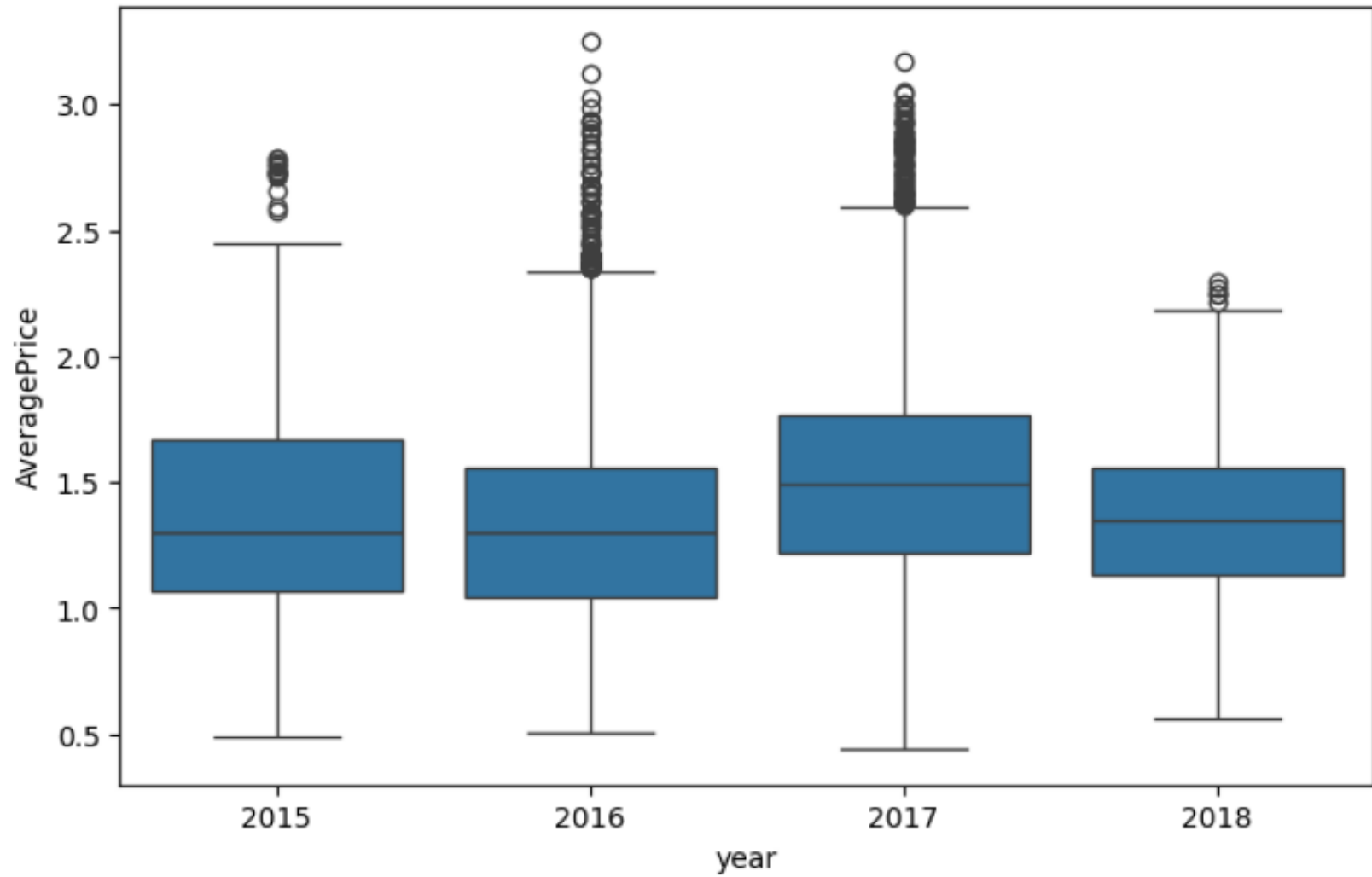


Precio promedio por tipo de aguacate



Aguacate - price_by_year.png

Precio promedio por año



Modelo: Bitcoin

Se predice la dirección del precio de Bitcoin (sube o baja al día siguiente) usando indicadores como precio de apertura, cierre, volumen, medias móviles y volatilidad. Modelo: Random Forest Classifier.

Métricas del modelo Bitcoin

Unnamed: 0	precision	recall	f1-score	support
0	0.5806451612903226	0.3857142857142857	0.463519313304721	140.0
1	0.4941176470588235	0.6829268292682927	0.5733788395904437	123.0
accuracy	0.5247148288973384	0.5247148288973384	0.5247148288973384	0.5247148288973384
macro avg	0.537381404174573	0.5343205574912893	0.5184490764475823	263.0
weighted avg	0.5401779207942222	0.5247148288973384	0.5148984833927206	263.0

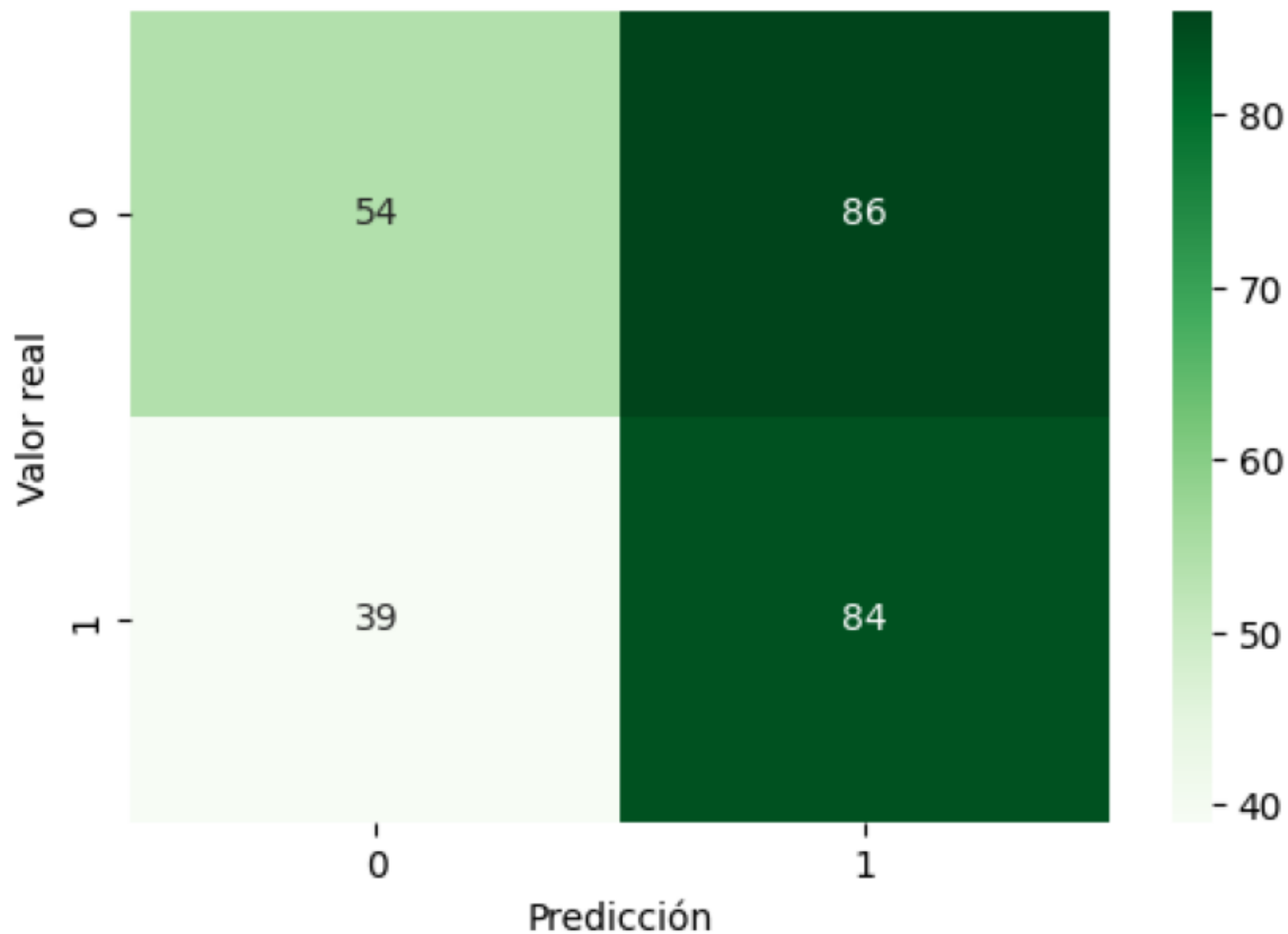
Bitcoin - 1_evolucion_precio.png

Evolución del precio de cierre de Bitcoin

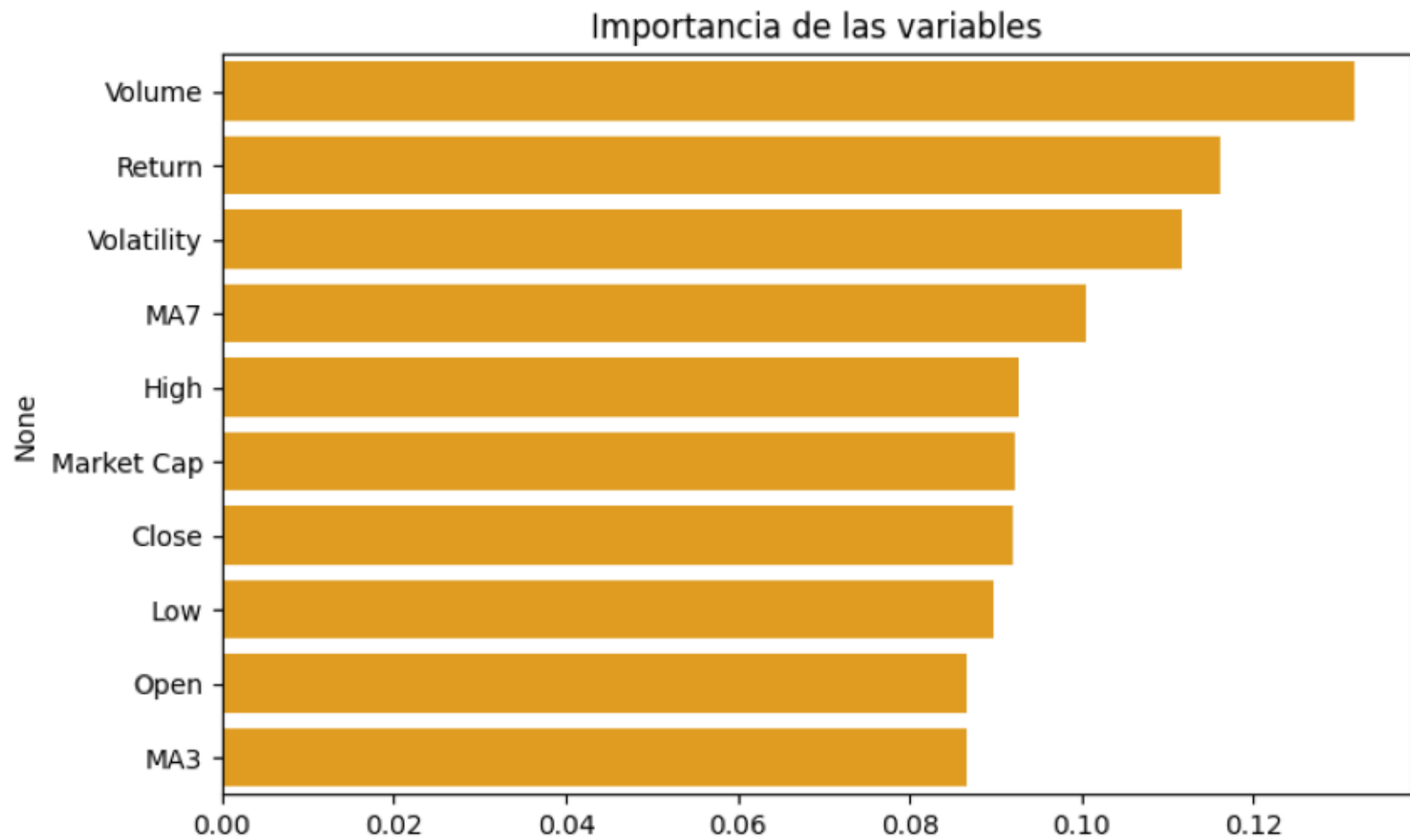


Bitcoin - 2_matriz_confusion.png

Matriz de confusión



Bitcoin - 3_importancia_variables.png



Modelo: BodyFat

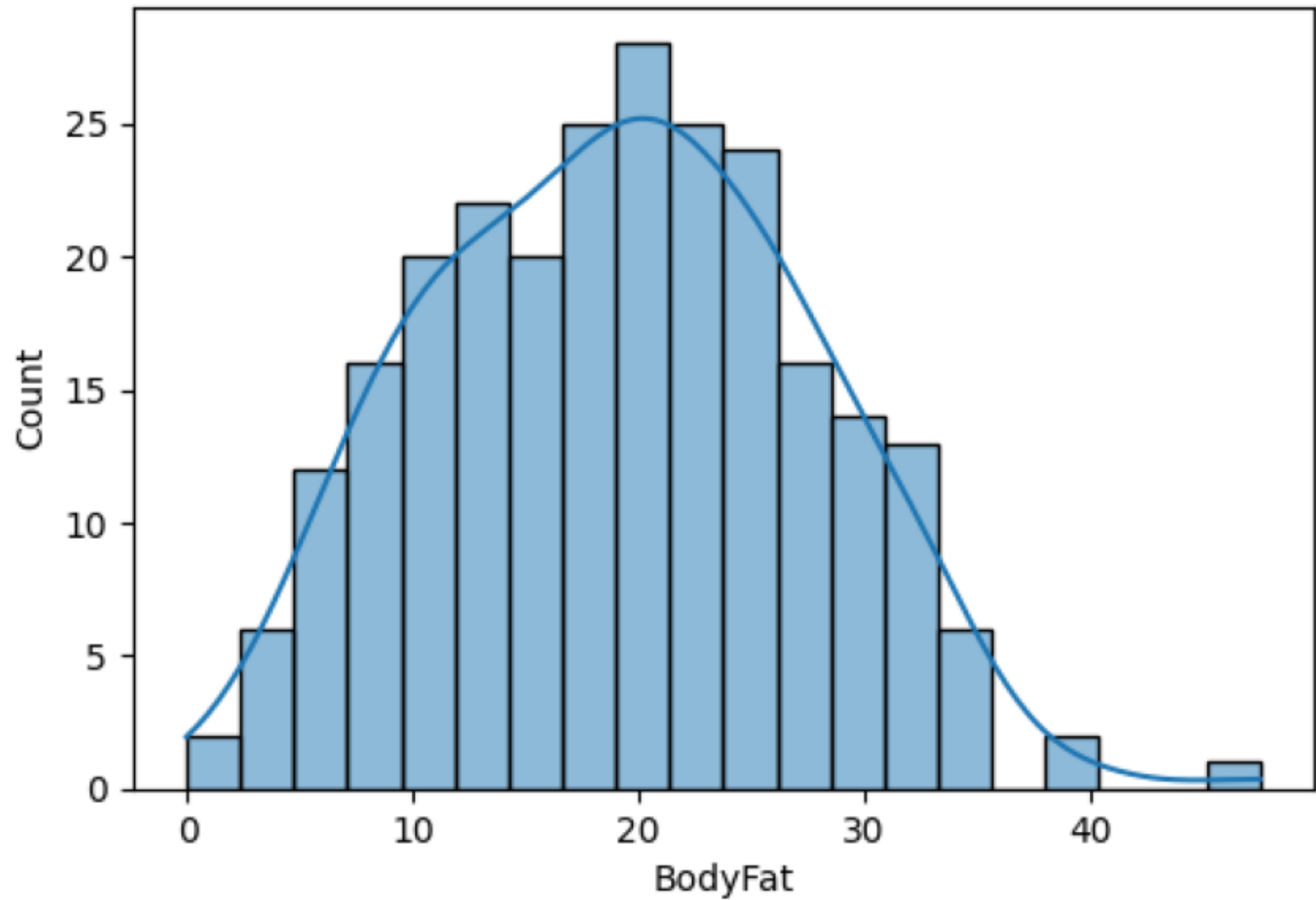
Se predice el porcentaje de grasa corporal (BodyFat) usando medidas antropométricas del cuerpo (edad, peso, altura, circunferencias, etc.), excluyendo la densidad. Modelo: Random Forest Regressor.

Métricas del modelo BodyFat

Métrica	Valor
MAE	3.9586470588235314
MSE	24.774759578431404
R ²	0.6187835600104352

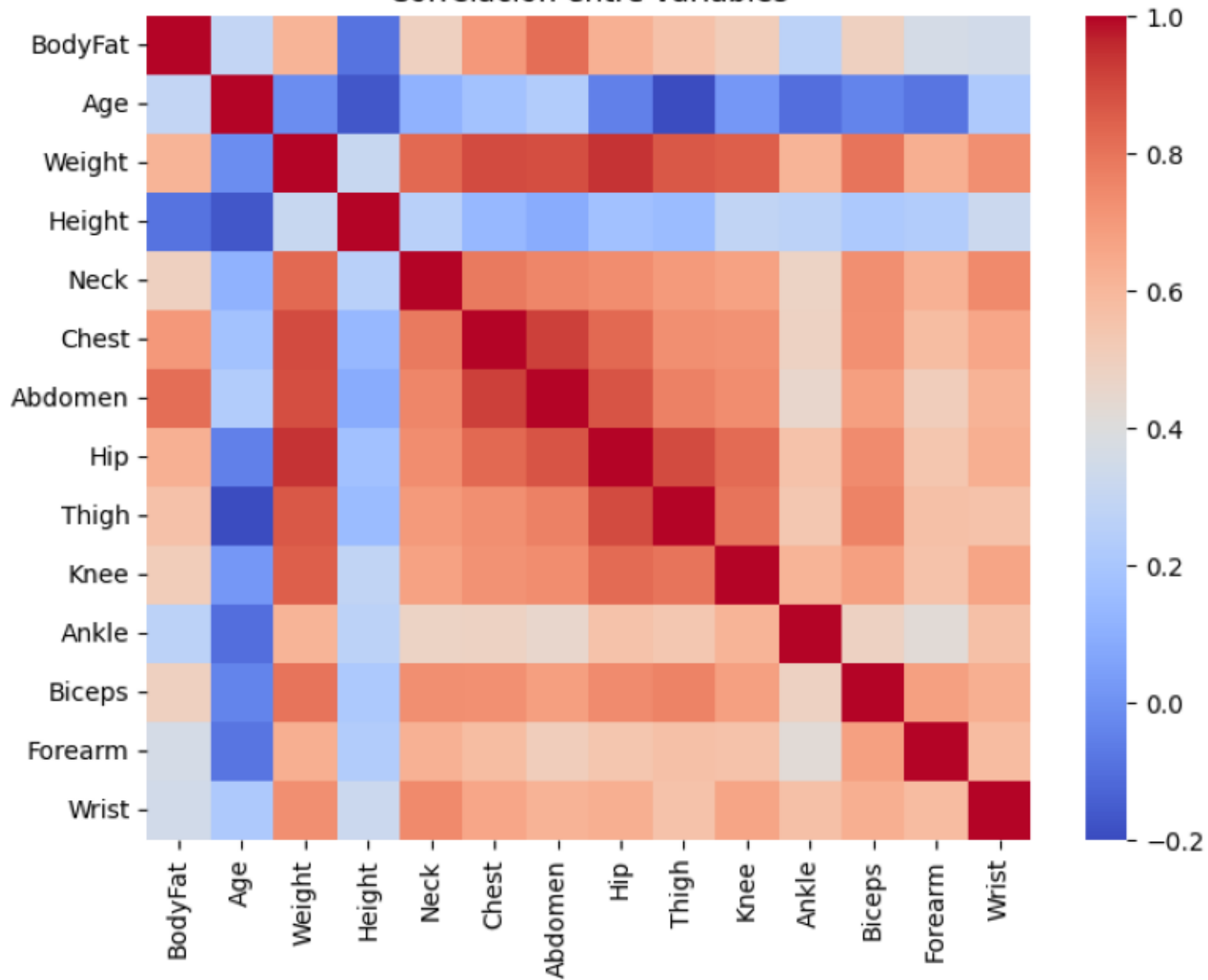
BodyFat - bodyfat_distribution.png

Distribución del BodyFat

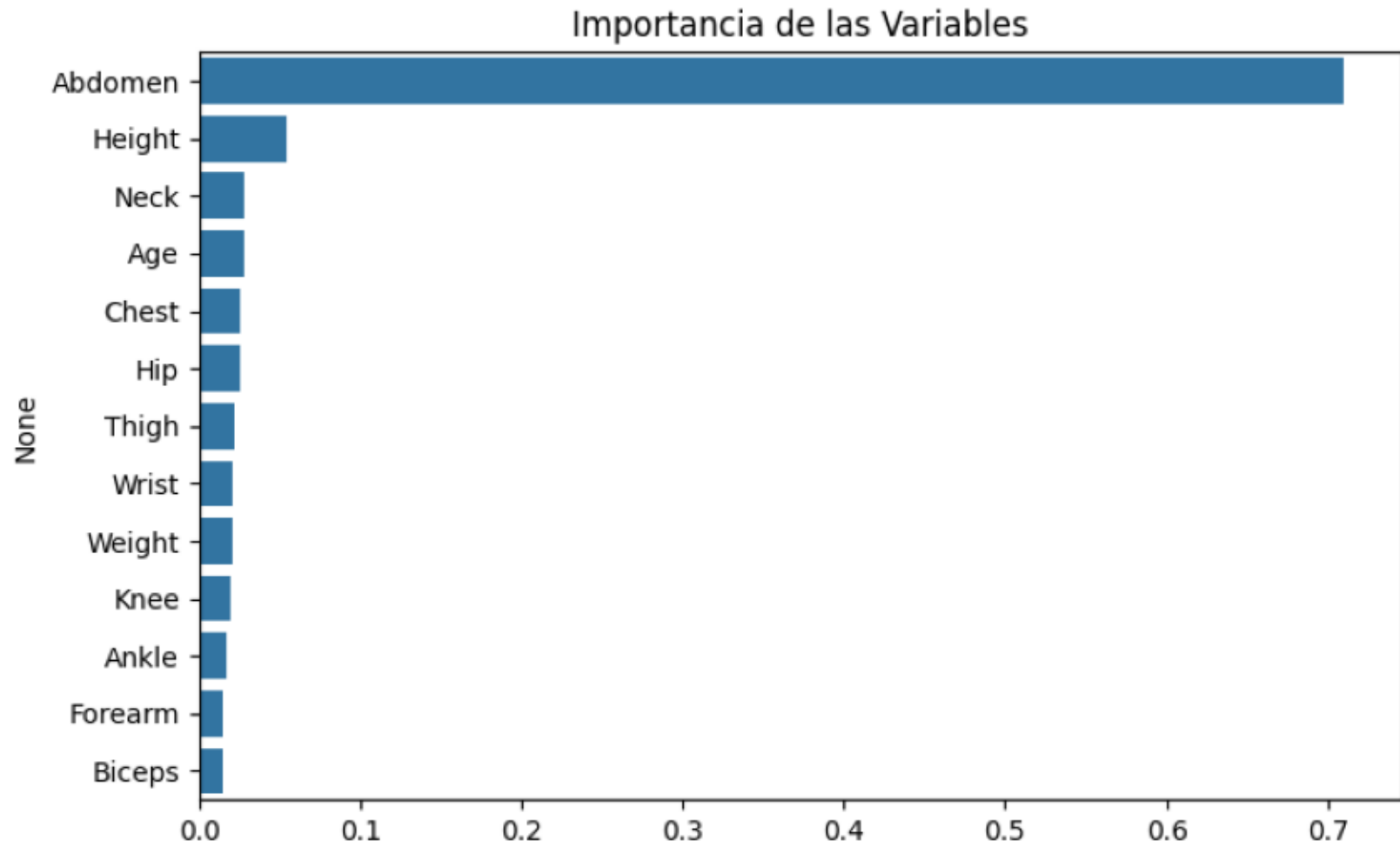


BodyFat - correlation_matrix.png

Correlación entre variables

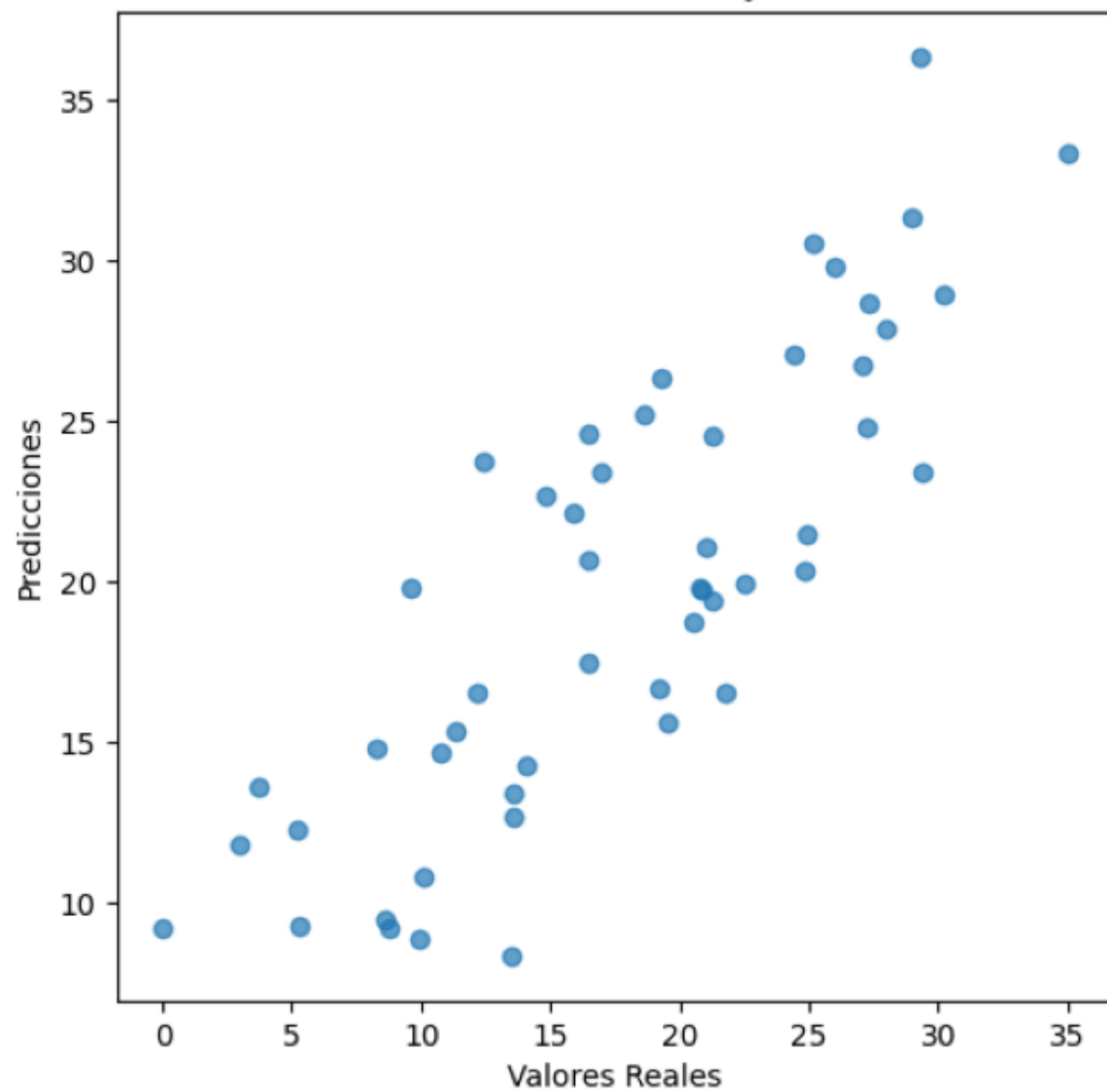


BodyFat - feature_importance.png



BodyFat - pred_vs_real.png

Predicción del BodyFat



Modelo: Telecom Churn

Se predice si un cliente de telecomunicaciones abandonará el servicio (Churn) según características demográficas, de contrato y uso de servicios. Modelo: Random Forest Classifier.

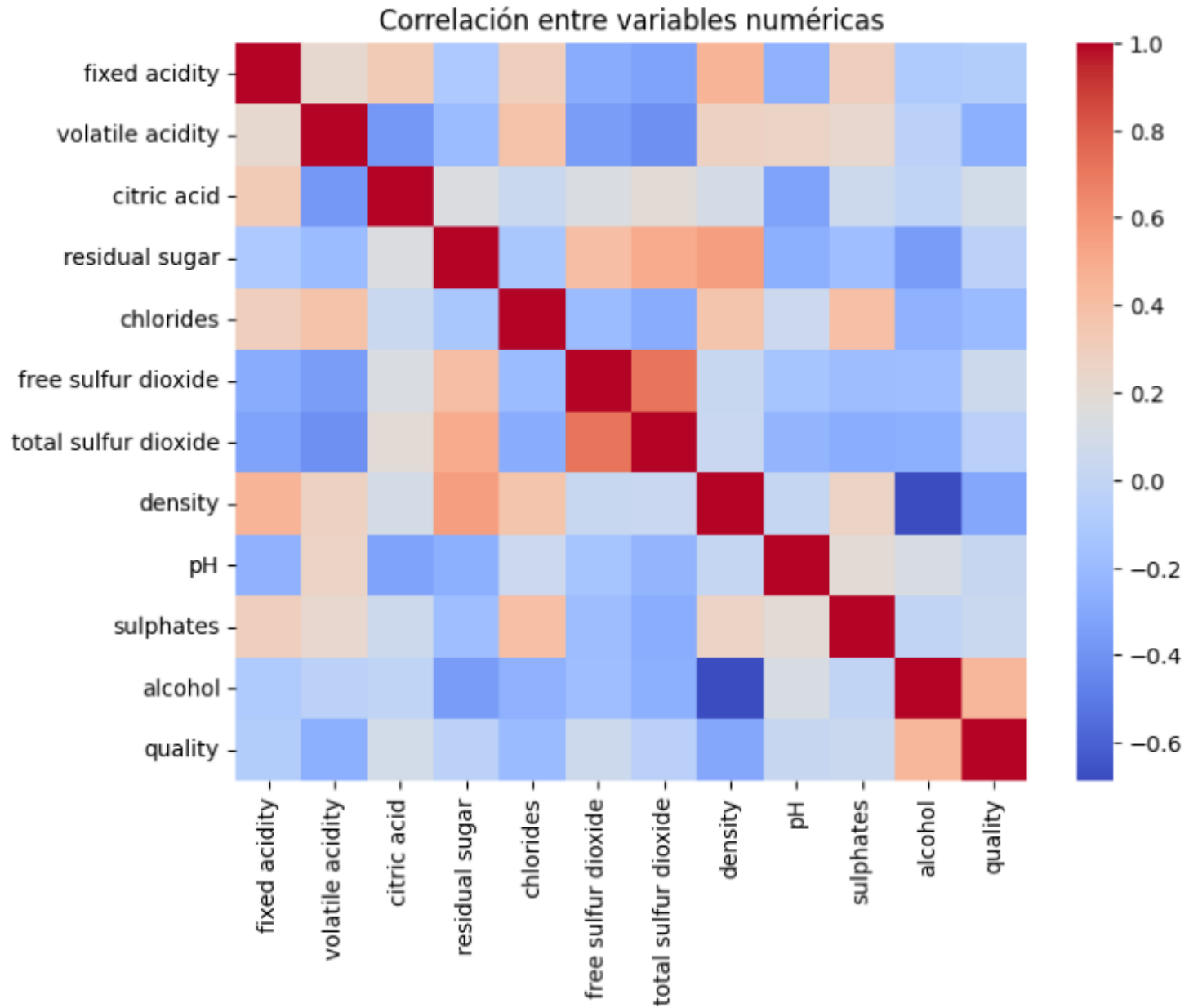
Modelo: Vino

Se predice la calidad del vino según sus propiedades químicas (acidez, azúcar, pH, alcohol, etc.) y tipo de vino. Modelo: Random Forest Regressor.

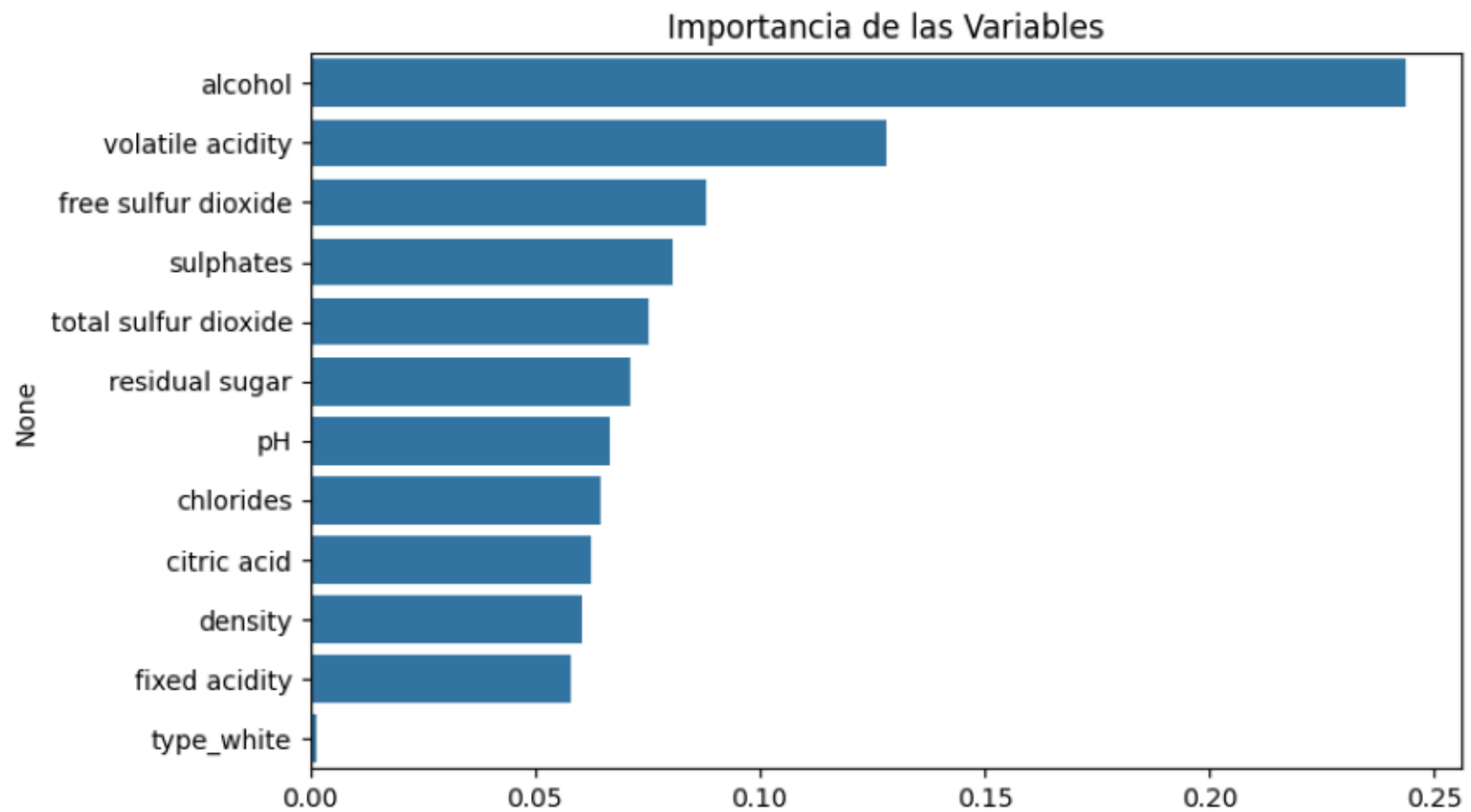
Métricas del modelo Vino

Métrica	Valor
MAE	0.4028692307692307
MSE	0.3142512692307692
R ²	0.5603643965162568

Vino - correlation_matrix.png

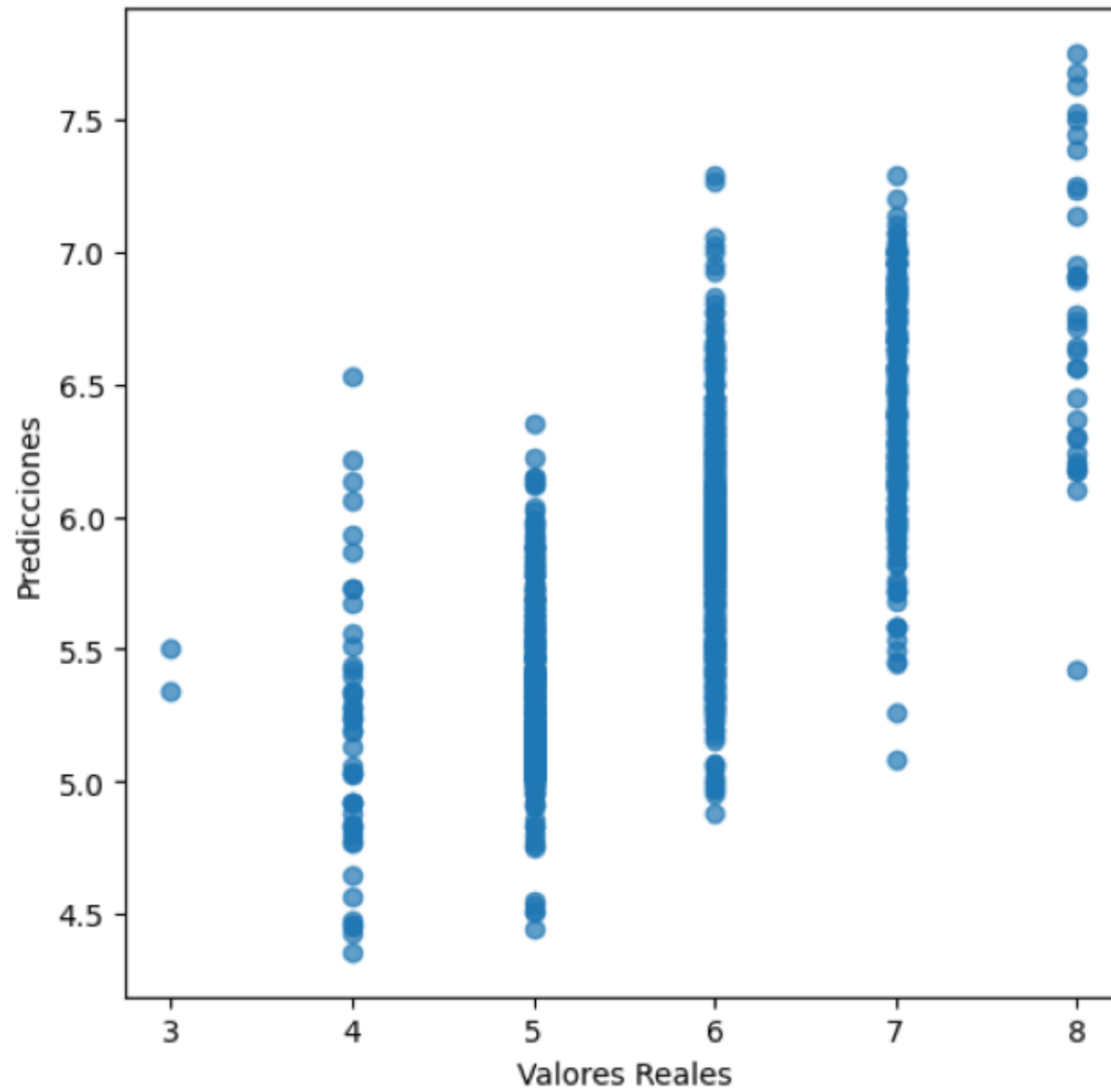


Vino - feature_importance.png



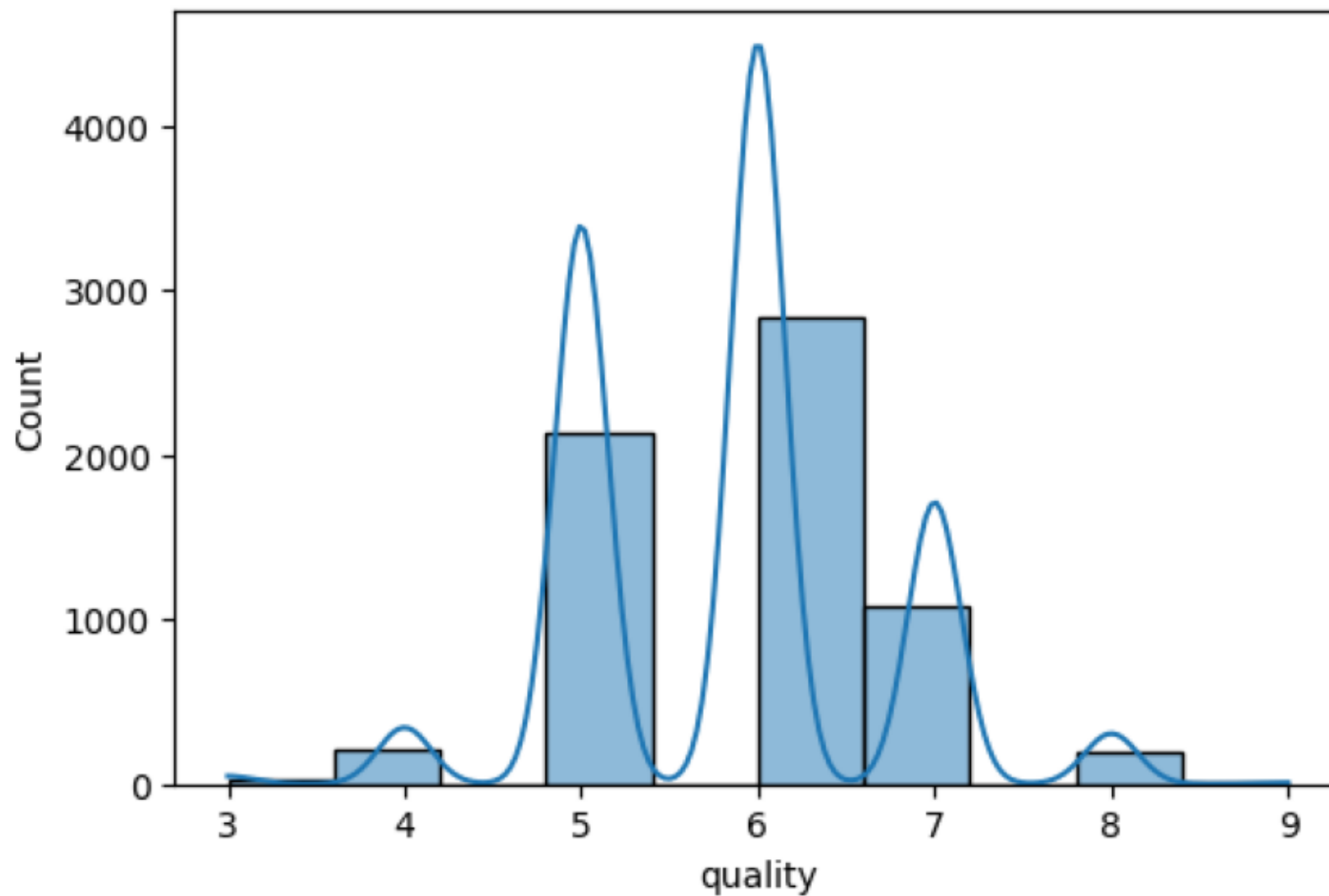
Vino - pred_vs_real.png

Predicción de calidad del vino



Vino - quality_distribution.png

Distribución de la calidad del vino



Código Python - Aguacate (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import os

# Cargar datos
data = pd.read_csv("data/aguacate.csv")

# Crear carpeta para figuras
os.makedirs("figures_aguacate", exist_ok=True)

# Visualización inicial
plt.figure(figsize=(8,5))
sns.histplot(data["AveragePrice"], bins=30, kde=True, color='green')
plt.title("Distribución del Precio Promedio del Aguacate")
plt.savefig("figures_aguacate/averageprice_distribution.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(8,5))
sns.boxplot(x="year", y="AveragePrice", data=data)
plt.title("Precio promedio por año")
plt.savefig("figures_aguacate/price_by_year.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(10,5))
sns.boxplot(x="type", y="AveragePrice", data=data)
plt.title("Precio promedio por tipo de aguacate")
plt.savefig("figures_aguacate/price_by_type.png", bbox_inches="tight")
plt.close()

# Preparación de datos
data = data.drop(columns=["id", "Date"])

label_encoders = {}
for col in ["type", "region"]:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le

X = data.drop(columns=["AveragePrice"])
y = data["AveragePrice"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Código Python - Aguacate (líneas 51-86)

```
# Modelo de IA
model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# Evaluación
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

results = pd.DataFrame({"Métrica": ["MAE", "RMSE", "R²"], "Valor": [mae, rmse, r2]})
results.to_csv("figures_aguacate/model_metrics.csv", index=False)
print("Evaluación del modelo:")
print(results)

# Visualización de resultados
plt.figure(figsize=(8,6))
sns.scatterplot(x=y_test, y=y_pred)
plt.xlabel("Valores reales")
plt.ylabel("Predicciones")
plt.title("Predicción vs Realidad - Precio de Aguacate")
plt.savefig("figures_aguacate/pred_vs_real.png", bbox_inches="tight")
plt.close()

# Importancia de variables
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
plt.figure(figsize=(8,5))
sns.barplot(x=importances.values, y=importances.index)
plt.title("Importancia de las variables en el modelo")
plt.savefig("figures_aguacate/feature_importance.png", bbox_inches="tight")
plt.close()

# Exportar modelo
joblib.dump(model, "modelo_precio_aguacate.pkl")
print("\nModelo exportado exitosamente a 'modelo_precio_aguacate.pkl'")
```

Código Python - Bitcoin (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import os
import joblib

# Crear carpeta para figuras
os.makedirs("figures_bitcoin", exist_ok=True)

# Cargar datos
data = pd.read_csv("data/bitcoin.csv")

# Limpiar columnas numéricas con comas y símbolos
for col in ["Volume", "Market Cap"]:
    data[col] = data[col].astype(str).replace({" ": ""}, regex=True)
    data[col] = data[col].replace(r"^[0-9\.\-]", "", regex=True)
    data[col] = pd.to_numeric(data[col], errors='coerce')

# Convertir fecha y ordenar
data["Date"] = pd.to_datetime(data["Date"])
data = data.sort_values("Date")

# Crear features
data["Return"] = data["Close"].pct_change()
data["MA3"] = data["Close"].rolling(3).mean()
data["MA7"] = data["Close"].rolling(7).mean()
data["Volatility"] = data["Close"].rolling(7).std()
data["Next_Close"] = data["Close"].shift(-1)
data["Direction"] = (data["Next_Close"] > data["Close"]).astype(int)
data = data.dropna()

# Variables predictoras y etiqueta
X = data[["Open", "High", "Low", "Close", "Volume", "Market Cap", "Return", "MA3", "MA7", "Volatility"]]
y = data["Direction"]

# División de datos
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=777, shuffle=True)

# Entrenamiento del modelo
model = RandomForestClassifier(n_estimators=200, random_state=777)
model.fit(X_train, y_train)

# Evaluación
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

Código Python - Bitcoin (líneas 51-83)

```
report_dict = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report_dict).transpose()
report_df.to_csv("figures_bitcoin/classification_report.csv")

print(f"Accuracy del modelo: {acc:.4f}")
print("\nReporte de clasificación:")
print(report_df)

# Visualizaciones
plt.figure(figsize=(10,5))
sns.lineplot(x="Date", y="Close", data=data)
plt.title("Evolución del precio de cierre de Bitcoin")
plt.savefig("figures_bitcoin/1_evolucion_precio.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Greens")
plt.title("Matriz de confusión")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.savefig("figures_bitcoin/2_matriz_confusion.png", bbox_inches="tight")
plt.close()

importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
plt.figure(figsize=(8,5))
sns.barplot(x=importances.values, y=importances.index, color='orange')
plt.title("Importancia de las variables")
plt.savefig("figures_bitcoin/3_importancia_variables.png", bbox_inches="tight")
plt.close()

# Guardar modelo entrenado
joblib.dump(model, "modelo_direccion_bitcoin.pkl")
print("\nModelo exportado a 'modelo_direccion_bitcoin.pkl' y figuras guardadas en /figures_bitcoin")
```

Código Python - BodyFat (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import os

# Cargar datos
df = pd.read_csv("data/bodyfat.csv")

# Crear carpeta para figuras
os.makedirs("figures_bodyfat", exist_ok=True)

# Visualización inicial
plt.figure(figsize=(8,6))
sns.heatmap(df.drop("Density", axis=1).corr(), annot=False, cmap="coolwarm")
plt.title("Correlación entre variables")
plt.savefig("figures_bodyfat/correlation_matrix.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(6,4))
sns.histplot(df["BodyFat"], bins=20, kde=True)
plt.title("Distribución del BodyFat")
plt.savefig("figures_bodyfat/bodyfat_distribution.png", bbox_inches="tight")
plt.close()

# Preparación de datos
X = df.drop(["BodyFat", "Density"], axis=1)
y = df["BodyFat"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=777)

# Entrenamiento del modelo
model = RandomForestRegressor(n_estimators=200, random_state=777)
model.fit(X_train, y_train)

# Evaluación del modelo
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

results = pd.DataFrame({
    "Métrica": ["MAE", "MSE", "R²"],
    "Valor": [mae, mse, r2]
```

Código Python - BodyFat (líneas 51-74)

```
)  
results.to_csv("figures_bodyfat/model_metrics.csv", index=False)  
print("Resultados del modelo:")  
print(results)  
  
# Visualización de resultados  
plt.figure(figsize=(6,6))  
plt.scatter(y_test, y_pred, alpha=0.7)  
plt.xlabel("Valores Reales")  
plt.ylabel("Predicciones")  
plt.title("Predicción del BodyFat")  
plt.savefig("figures_bodyfat/pred_vs_real.png", bbox_inches="tight")  
plt.close()  
  
plt.figure(figsize=(8,5))  
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)  
sns.barplot(x=importances.values, y=importances.index)  
plt.title("Importancia de las Variables")  
plt.savefig("figures_bodyfat/feature_importance.png", bbox_inches="tight")  
plt.close()  
  
# Guardar modelo entrenado  
joblib.dump(model, "bodyfat_model.pkl")  
print("\nModelo exportado exitosamente a 'bodyfat_model.pkl'")
```

Código Python - Telecom Churn (líneas 1-50)

```
# Importar librerías

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import joblib
import os

# Cargar datos

df = pd.read_csv("data/telecomunicaciones.csv")

# Crear carpeta para figuras

os.makedirs("figures_telecomunicaciones", exist_ok=True)

# Preprocesamiento

df = df.drop("customerID", axis=1)
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors='coerce')
df = df.dropna()

categorical_cols = df.select_dtypes(include="object").columns
le = LabelEncoder()
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

X = df.drop("Churn", axis=1)
y = df["Churn"]

# División de datos

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Entrenamiento del modelo

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
```

Código Python - Telecom Churn (líneas 51-83)

```
# Evaluación del modelo

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.to_csv("figures_telecomunicaciones/classification_report.csv")

print("Accuracy del modelo:", acc)
print("\nReporte de clasificación:")
print(report_df)

plt.figure(figsize=(6,5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues")
plt.title("Matriz de Confusión")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.savefig("figures_telecomunicaciones/confusion_matrix.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(8,5))
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
sns.barplot(x=importances.values, y=importances.index)
plt.title("Importancia de las variables")
plt.savefig("figures_telecomunicaciones/feature_importance.png", bbox_inches="tight")
plt.close()

# Guardar modelo entrenado

joblib.dump(model, "telecomunicaciones_model.pkl")
```

Código Python - Vino (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import os

# Cargar datos
df = pd.read_csv("data/vino.csv")

os.makedirs("figures_vino_quality", exist_ok=True)

plt.figure(figsize=(8,6))
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=False, cmap="coolwarm")
plt.title("Correlación entre variables numéricas")
plt.savefig("figures_vino_quality/correlation_matrix.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(6,4))
sns.histplot(df["quality"], bins=10, kde=True)
plt.title("Distribución de la calidad del vino")
plt.savefig("figures_vino_quality/quality_distribution.png", bbox_inches="tight")
plt.close()

# Preparación de datos
X = df.drop("quality", axis=1)
X = pd.get_dummies(X, drop_first=True) # Codifica 'type' y otras categóricas
y = df["quality"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenamiento del modelo
model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# Evaluación del modelo
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
results = pd.DataFrame({"Métrica":["MAE", "MSE", "R²"], "Valor":[mae, mse, r2]})
results.to_csv("figures_vino_quality/model_metrics.csv", index=False)
print("Resultados del modelo:")
print(results)
```

Código Python - Vino (líneas 51-69)

```
# Visualización de resultados
plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.xlabel("Valores Reales")
plt.ylabel("Predicciones")
plt.title("Predicción de calidad del vino")
plt.savefig("figures_vino_quality/pred_vs_real.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(8,5))
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
sns.barplot(x=importances.values, y=importances.index)
plt.title("Importancia de las Variables")
plt.savefig("figures_vino_quality/feature_importance.png", bbox_inches="tight")
plt.close()

# Guardar modelo entrenado
joblib.dump(model, "vino_quality_model.pkl")
```